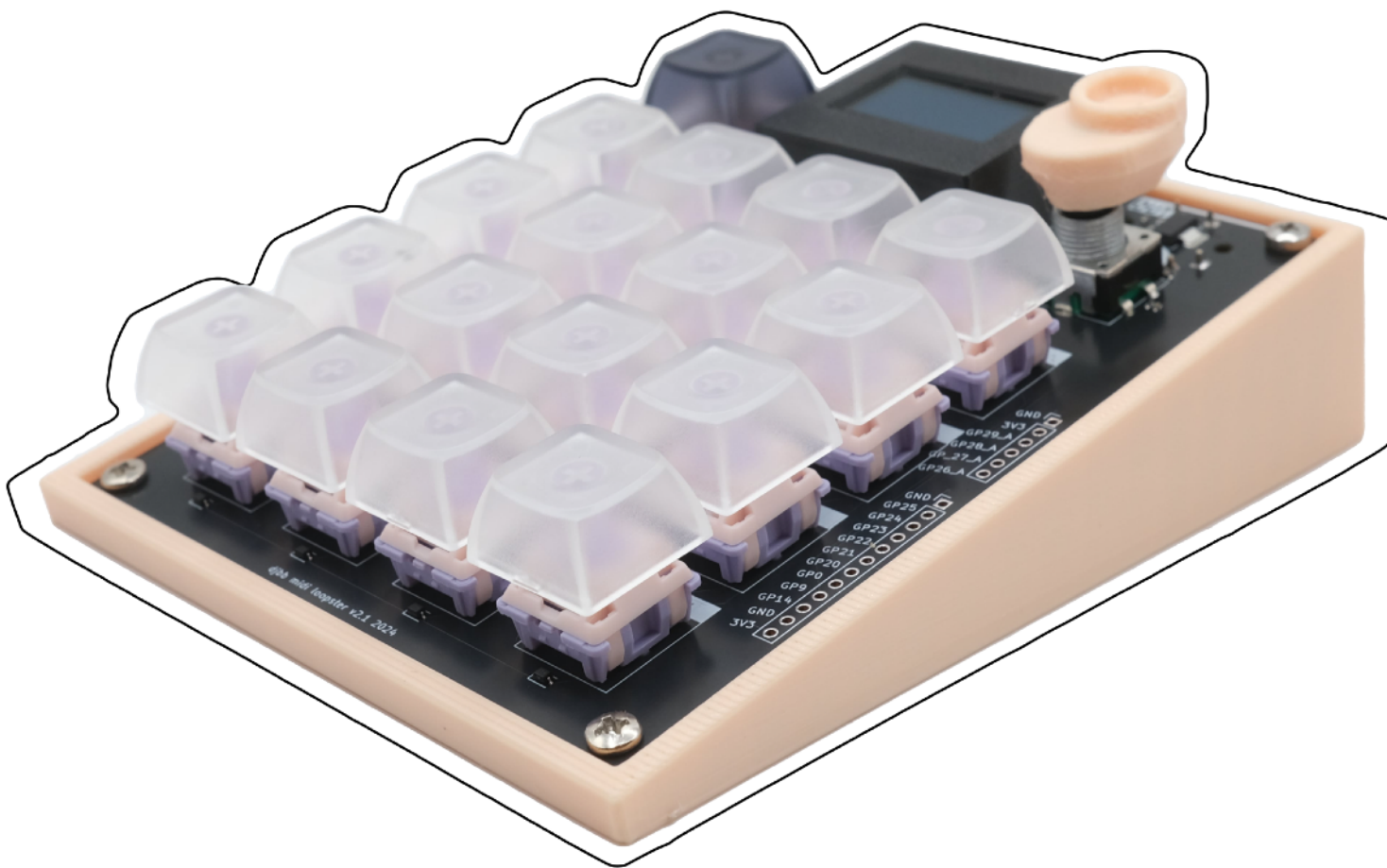


DJBB

MIDI LOOPSTER 2

User Manual

Last Updated: 12/05/2024



The DJBB Midi Loopster 2 is an open source midi controller with looping functionality. It features a compact design, full sized midi I/O + USB midi, per-key RGB LEDs to help you keep track of what's going on.

Features:

- Full-sized MIDI In and Out & USB MIDI (micro usb)
- MIDI Looper - record and overdub loops of midi notes
- Mechanical keyswitches that aren't too loud.
- Scale select
- Unique encoder based arpeggiator
- Presets - Save and load settings (like scale, midi channels, startup screen, and more)
- Set velocity of pads independently
- Velocity map a single pad across all pads - nice for hi-hats
- Open Source & easy to extend via extra GPIOs
- "Chord Mode" - record a chord or sequence to a specific pad and play it back as a loop or 1-shot

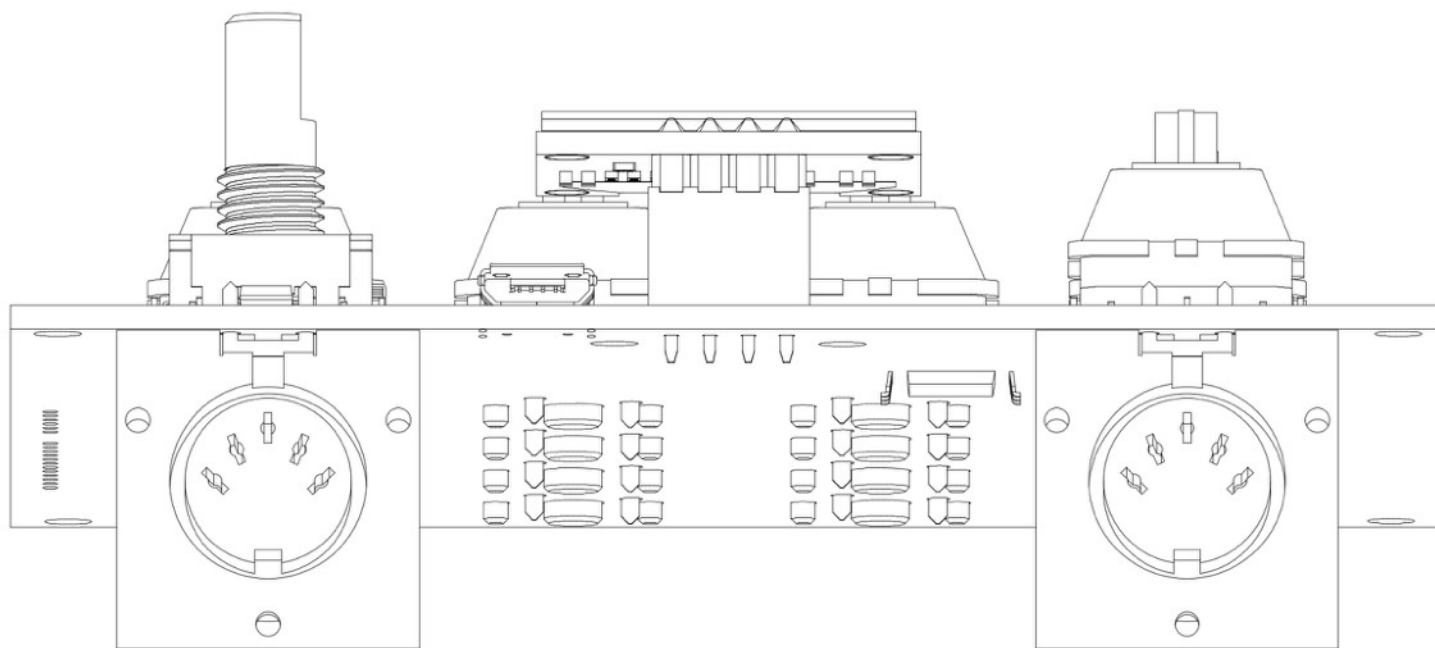


Table of Contents

Links & Resources	4
Play Screen Quick Reference	4
Using the Loopster	5
1) Play Screen	5
2) Scale Screen	8
3) Looper Screen	8
4) MIDI Settings	9
5) Other Settings	9
6) Load Preset	9
7) Save Preset	9
Installing Software / Updating	10
Feedback / Reporting Issues	11
Technical	12
Modifying Code	12
Extending the Loopster 2	12
useraddons.py	13
Appendix A: List of Settings	15
Appendix B: Select User Add-ons Examples	17

Links & Resources

Github: <https://github.com/derrickthomin/DJBB-Loopster-2.0>

STL Files: <https://www.printables.com/model/1066847-djbb-midi-loopster-20-case-and-screen>

Submit Issue / Idea: <https://github.com/derrickthomin/DJBB-Loopster-2.0/issues>

Play Screen Quick Reference

This only covers the basic actions on the play screen (default startup screen) and is intended to get you looping pronto. See the rest of the guide for more details.

To Change Play Modes: double click [fn]. Modes are (CHD) (VEL) and (ARP)

Play Mode	Desired Action	Buttons
(CHD)	Record notes to a pad	Hold [fn] and press a pad to begin recording notes. Click [fn] to stop recording
(CHD)	Change from loop to 1-shot	Hold pad with recorded notes and turn the encoder
(CHD)	Change record quantization settings	Hold [fn] and turn encoder to change value Hold encoder button and turn to change amount.
(VEL)	Set velocity of individual pad(s)	Hold pad(s) and turn encoder
(VEL)	Map one note across 16 velocities	Hold [fn] and click a pad. Do the same to undo.
(ARP)	Cycle through arp notes	Hold pads and turn the encoder. Pads with chords recorded will add all notes in the chord to the arp sequence
(ARP)	Change arpeggiator settings	Hold [fn] and turn encoder to change arp type Hold encoder button and turn to change length (gate)

Using the Loopster

To change settings on the current screen, turn the encoder. To move between screens click the encoder, then twist. The **NAV** icon on the bottom of the screen indicates that turning the encoder switches screens. The encoder LED will also light up white.

To lock the encoder so that you don't accidentally change something, double click the encoder button. This can be useful when using the arpeggiator and other functions.

1) Play Screen

This is the default startup screen. Here you can Change MIDI banks, change play modes, and configure quantization and arpeggiator settings. By default, the Loopster 2 starts in Chord Mode. You can see the current play mode by looking at the bottom of the screen.



Play mode screen with the ARP play mode.

Play Modes

Double click the [fn] (top left) button to change play modes. The change will be reflected on the bottom of the screen

(CHD) Chord Mode

This is the default mode. In this mode you can record MIDI loops to a specific pad for playback.

Hold [fn] and click a pad to begin recording. To stop recording, click [fn] again. To change record quantization time, hold [fn] and turn the encoder.

To change quantization amount, hold the encoder button and turn the encoder. See additional recording settings in the **MIDI Settings** section.

To change the recorded chord or sequence from a loop to a one-shot, hold the pad and turn the encoder. See the **Other Settings** section for how to change which is selected by default for new recordings.

To start/stop a chord loop or trigger a one-shot, press the button with the recorded MIDI. Buttons with recorded MIDI will be purple, and green when currently playing.

(VEL) Velocity Mode

In VEL mode, holding the [fn] button and pressing a pad will map that MIDI note across all 16 pads with a gradient of velocities from 1-127. To undo, hold [fn] and click any pad.

To change the velocity for a specific pad, hold that pad (or multiple) and turn the encoder button.



*Hold **[fn]** and press pad to
velocity map single note
across all pads.*

(ARP) Arpeggiator Mode

In ARP mode, you can still play the pads as usual, but if you hold a pad and turn the encoder, it sends notes. If you hold a pad with MIDI recorded to it, all of the notes recorded are added to the arpeggio.

To change the arpeggiator direction, hold [fn] and turn the encoder. To change the length of each arp note (gate), hold the encoder button and turn the encoder.

Note that when switching to the arpeggiator mode, the encoder is locked. Holding pads and turning it will still play notes, but turning it on its own will do nothing. This is to prevent accidental bank/screen changes when using ARP mode. Double click the encoder button to unlock. Changing the play mode will also unlock it (double click [fn])



*Play screen with arpeggiator
mode selected*

2) Scale Screen

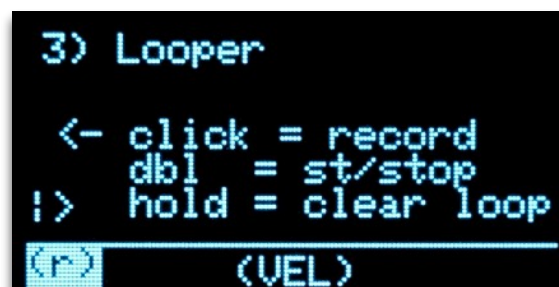
Turn the encoder to change the scale. Click the [fn] button, or hold [fn] and turn the encoder to change the root note.



Scale screen

3) Looper Screen

- To Begin Recording or Stop Recording, click [fn]
- To Start or Stop loop, double click [fn]
- To Overdub click [fn] while the loop is playing. Click again to disarm recording.
- To Delete The Loop, hold [fn]
- To Delete a Random Loop Note, turn encoder counterclockwise
- Only <DJT MAX?????> notes can exist at a time



Looper currently recording

4) MIDI Settings

All of the settings in on this screen will be saved and loaded with presets.

Turn the encoder to change the value of the setting. Hold **[fn]** and turn the encoder to scroll through settings.

See [Appendix A](#) for the full list of settings

5) Other Settings

All of the settings in on this screen will be saved and loaded with presets. Note that some can also be changed via other menus (like quantization and arpeggiator settings).

See [Appendix A](#) for the full list of settings

6) Load Preset

Use the encoder to select a preset. Press [fn] to load it.

7) Save Preset

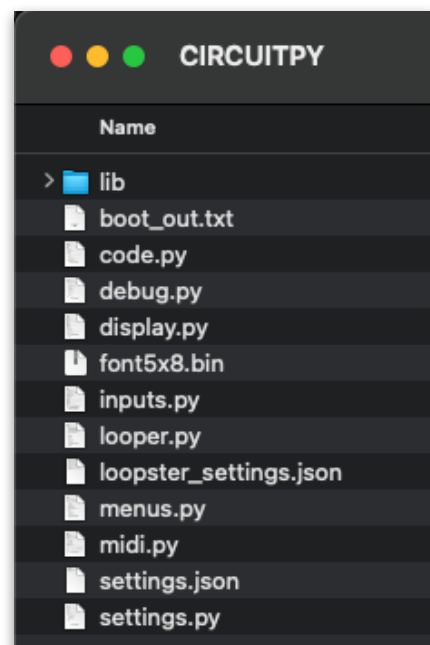
Use the encoder to select a preset, or select *NEW*. Press [fn] to overwrite/save.

Installing Software / Updating

The Loopster 2 runs on CircuitPython¹, a lightweight version of the Python programming language. This is the “operating system”, if you will.

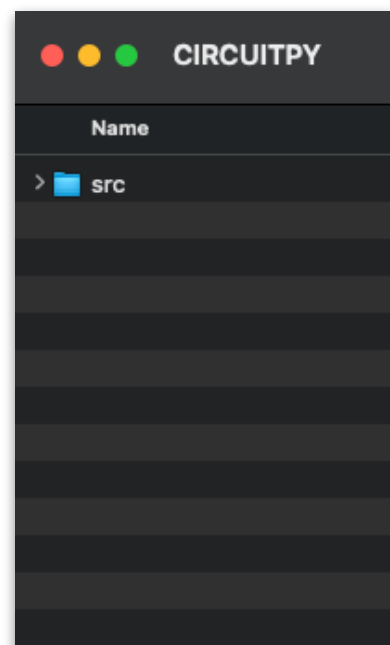
1. Go to the Loopster [2 GitHub repo](#) and download it as a ZIP file.
2. Hold the [fn] button <DJT OR NE>> and continue holding it while plugging in the Loopster 2. It should show up just like a USB drive on your computer. If not, make sure the USB cable is not power only.
3. Open the **src** folder and drag all files/folders to the Loopster 2 root directory. If prompted to overwrite files, select yes.

YES



Copy **contents** of **src** folder...

NO



... not the whole folder

¹You can read more about installing CircuitPython on the Pico here: <https://learn.adafruit.com/getting-started-with-raspberry-pi-pico-circuitpython/circuitpython>

Feedback / Reporting Issues

If you find any bugs or issues, please report them via the GitHub link below. If you have any enhancement / new feature ideas, I'd also love to hear!

Report Issues and Ideas Here: [https://github.com/derrickthomin/DJBB-Loopster-2.0/
issues](https://github.com/derrickthomin/DJBB-Loopster-2.0/issues)

Technical

Only read on if you are interested in modifying the Loopster and digging into the code. Otherwise, just go have fun with it.

Modifying Code

To be able to save updates to the loopster, it must be in “read only” mode. More info can be [found here](#).

To start the Loopster 2 in read-only mode, hold the [fn] button, and plug it into your computer while continuing to hold. Once it boots up, you should be able to move files to/from the Loopster 2 just like a USB drive. Note that in read only mode, presets cannot be saved (again, see the link listed above).

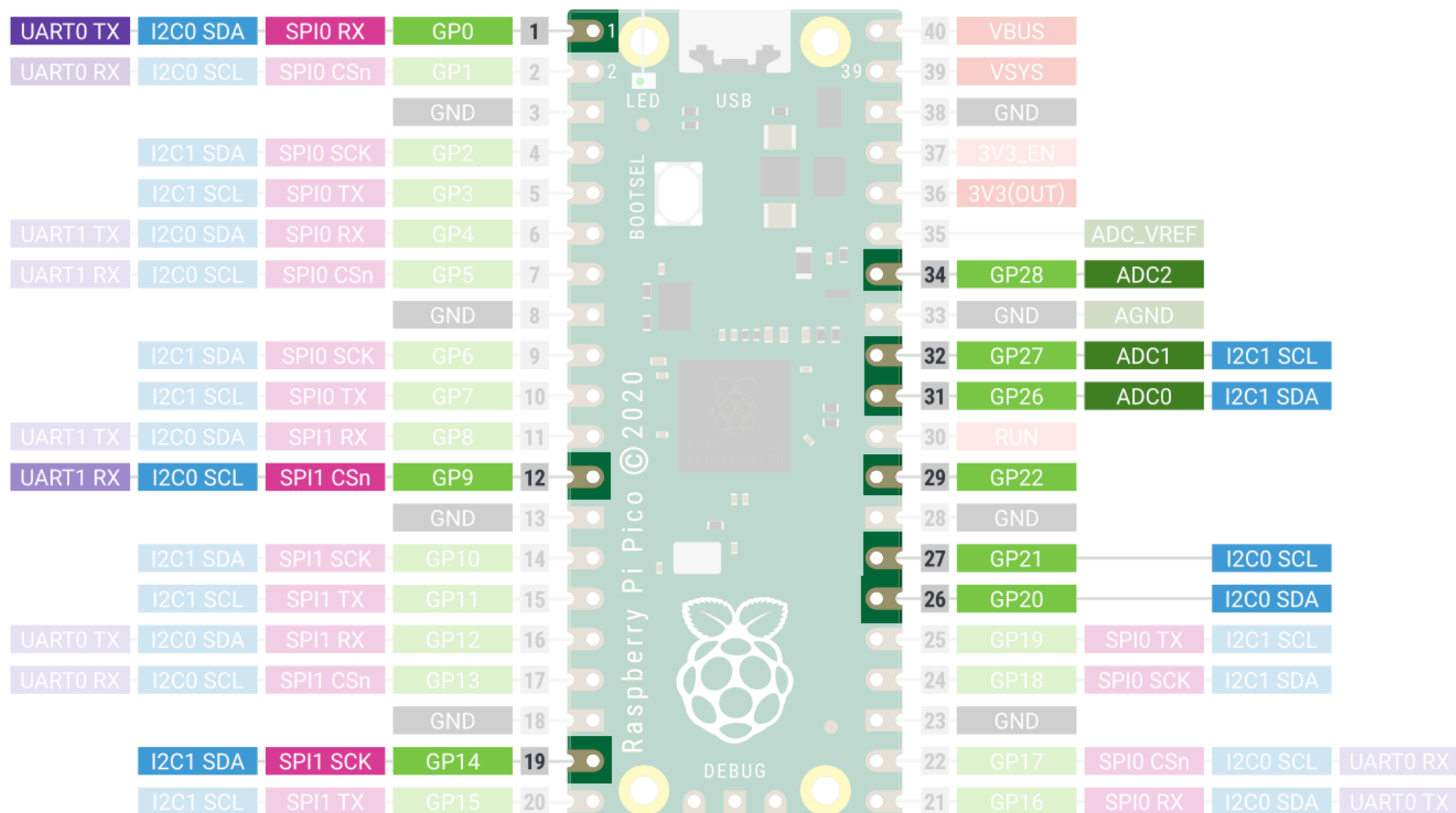
Extending the Loopster 2

[Here is a YouTube video with some examples of what you can do](#). Code snippets for each of these add ons can be found in useraddons.py in the GitHub src folder. If you’d like, you can 3D print the breadboard edition of the case - [find the STL file here](#).

The Loopster 2 has 13 (4 are analog) GPIO pins broken out on the right side of the board. These can be used to add your own peripherals to the Loopster, like additional knobs or buttons, LEDs, and pretty much whatever you want. The Loopster is built on the RP2040 MCU by the Raspberry Pi foundation. Anything you can do with that in CircuitPython, you could add onto the Loopster 2.

For reference, there is a pinout diagram of the Raspberry Pi Pico (the official board for the RP2040) with the available pins highlighted. Note that more pins are available than are show in the diagram.

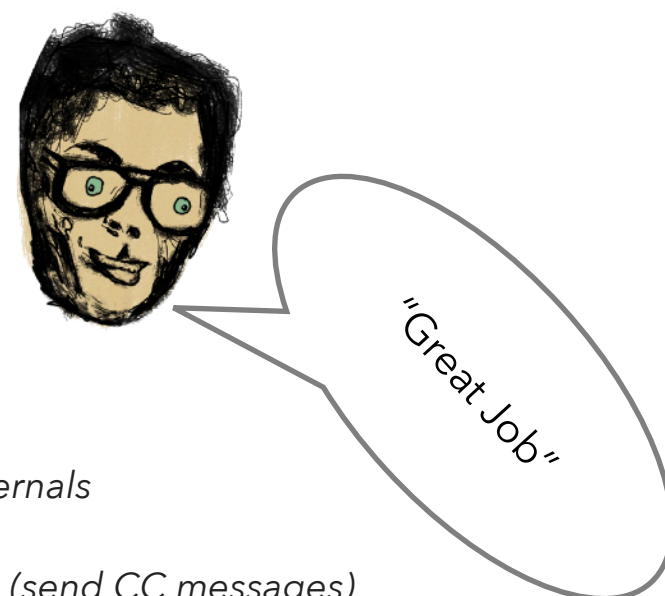
Available Digital Pins	Available Analog Pins
GP0, GP9, GP14, GP20, GP21, GP22, GP23, GP24, GP25	GP26, GP27, GP28, GP29



useraddons.py

This is the easiest place to start when extending your Loopster 2. Here you will find commented-out examples for...

- Neopixels*
- XY Joystick*
- Button for Shifting Octaves*
- Potentiometer for changing MIDI velocities*
- Encoder for changing MIDI channels*
- Photoresistor for changing MIDI velocities*
- Control a Motor using PWM*
- Play sounds with a piezo via PWM*
- Trigger relay switches to power high powered externals*
- PIR sensor to shift octaves*
- Accelerometer / Gyroscope to control modulation (send CC messages)*
- Update a 7 segment display*
- Use a DC motor as an input*



To add your own externals, write a function in `useraddons.py`, and then call that function in one of the “hooks” below. Each of these “hook” functions are already called in the main Loopster 2 code, so you don’t need to modify any other file to run your custom code. Again, see `useraddons.py` for a bunch of examples.

Here are the 4 “hooks” where you can call your custom code

Hook	Description	Example Use Cases
<code>check_addons_slow()</code>	Less frequent calls for non-urgent tasks.	- check a pot to change global velocity - Update an external display
<code>check_addons_fast()</code>	ore frequent calls for time-sensitive tasks.	- shift all midi notes when a button is pressed - change MIDI channel quickly (to get wacky with loops)
<code>handle_new_notes_on(noteval, velocity, padidx)</code>	Triggered when a new note is played. Your function does not need to use them, but these parameters are available for use: noteval: midi value (0-127) velocity: midi velocity (0-127) padidx: index of pad (0-15)	- light up neopixels when a note plays - Play note on piezo buzzer
<code>handle_new_notes_of(noteval, velocity, padidx)</code>	Same as above, but on note off	Same as above, but on note off

Appendix A: List of Settings

Below are the settings that are saved and loaded with presets. These can also be modified via presets.json. If an invalid value or setting is entered, it will be ignored with no warning, falling back to the default. See settings.py for the defaults.

Setting	Values	Description
debug	`True`, `False`	Flag indicating whether debug mode is enabled.
performance_mode	`True`, `False`	Flag indicating whether performance mode is enabled.
midibank_idx	Integer	The default MIDI bank index.
midi_channel_out	0-15	The MIDI channel to use for output.
midi_channel_in	0-15	The MIDI channel to use for input.
default_velocity	0-127	The default velocity value for MIDI notes.
default_bpm	Integer	The default BPM (beats per minute).
midi_notes_default	List of integers	The default MIDI notes.
midi_type	"ALL"	The type of MIDI connection.
midi_usb_io	"both", "in", "out"	The USB MIDI input/output configuration.
midi_aux_io	"both", "in", "out"	The auxiliary MIDI input/output configuration.
scale_idx	Integer	The default scale index.
rootnote_idx	Integer	The default root note index.
scalenotes_idx	Integer	The default scale notes index.
playmode	"chord"	The starting play mode.
midi_sync	`True`, `False`	Flag indicating the MIDI sync status.
midi_settings_page_indices	List of integers	The MIDI settings page indices.
settings_menu_option_indices	List of integers	The settings menu option indices.
chordmode_looptype	"chordloop"	The type of loop in chord mode.
arpeggiator_type	"up"	The type of arpeggiator pattern.
arpeggiator_length	"1", "1/2", "1/4", "1/8", "1/16", "1/32", "1/64"	The length of each arpeggiator note.
encoder_steps_per_arpname	Integer	Number of encoder steps per arpeggiator note. Higher values require more turns for the next note.
arp_is_polyphonic	`True`, `False`	Flag indicating if the arpeggiator is polyphonic.

quantize_time	"none", "1/4", "1/8", "1/16", "1/32"	The quantization time for MIDI notes.
quantize_strength	0-100	The strength of quantization applied to MIDI notes.
quantize_loop	"none", "1", "1/2", "1/4", "1/8"	The quantization loop setting.
trim_silence_mode	"start", "end", "both", "none"	The mode for trimming silence from MIDI notes.
startup_menu_idx	Integer	The index of the startup menu.
led_pixel_brightness	0.0-1.0	The brightness of the LED pixels.

Appendix B: Select User Add-ons Examples

See **useraddons.py** in the /src folder for a full list.

Control External Neopixels Based on Note Played

Perhaps you an additional neopixel display for performance purposes, and want to display animations based on MIDI data. <DJBB HELP>

Define the neopixels

```
extra_neopixels = neopixel.NeoPixel(board.GP25, 16, brightness=0.8)
```

Turn them on and off via the provided new notes hooks

```
def handle_new_notes_on_extra_pixels(padidx):  
    extra_neopixels[padidx] = (255, 255, 255) # White  
  
def handle_new_notes_off_extra_pixels(padidx):  
    extra_neopixels[padidx] = (0, 0, 0) # Black / Off
```

Button for Shifting Octaves

Use a button to shift all notes up or down an octave in real-time.

```
button = digitalio.DigitalInOut(board.GP0)  
button.direction = digitalio.Direction.INPUT  
button.pull = digitalio.Pull.UP  
  
def shift_all_notes():  
    if button.value:  
        shift_all_notes_octaves("up", 1)  
    else:  
        shift_all_notes_octaves("down", 1)
```